Universidade de São Paulo
Escola de Artes, Ciências e Humanidades
Sistemas de Informação

Relatório do Exercício-Programa

ACH2034 - Organização e Arquitetura de Computadores I

Gustavo Neves Moraes — NUSP: 16986195
Vinícius Cunha Pereira — NUSP: 16973242

Profa. Gisele S. Craveiro

São Paulo
2026

Sumário

1 Organização e Arquitetura MIPS **2**

2 Descrição do problema e código alto nível da solução **2**
2.1 Descrição do problema 2
2.2 Algoritmo de solução 2

3 Código em Assembly desenvolvido **4**
3.1 Bloco principal (`main`) 4
3.2 Sub-rotina `mmc` 5
3.3 Sub-rotina `divide` 6

4 Explicação detalhada das instruções utilizadas no código feito pela equipe **7**

1 Organização e Arquitetura MIPS

Muito texto.

2 Descrição do problema e código alto nível da solução

2.1 Descrição do problema

O problema resolvido corresponde à Questão 8 da "Lista de Problemas"do EP.

O item (a) pede a implementação de uma função com assinatura `int divide (int *m, int *n, int d)`, que recebe três inteiros positivos e deve devolver 1 caso o divisor `d` divida pelo menos um dos valores apontados por `m` ou `n`, e 0 caso contrário. Além de retornar esses valores, sempre que `d` divide `*m`, o valor apontado é atualizado para o quociente da divisão, e o mesmo vale para `*n`.

O item (b) pede um programa que leia dois inteiros positivos `m` e `n` e, utilizando obrigatoriamente a função do item (a), calcule o mínimo múltiplo comum (MMC) entre eles.

2.2 Algoritmo de solução

A estratégia adotada para resolução do problema foi a de fatoração por tentativa. Testa-se sequencialmente cada divisor candidato `d`, começando em 2. Para cada `d`, chama-se `divide(&m, &n, d)`. Se a chamada indicar que pelo menos um dos dois valores foi dividido, o divisor `d` é incorporado ao acumulador do MMC (multiplicando-o) e o mesmo `d` é testado novamente, pois ele pode dividir novamente algum dos dois valores restantes. Caso `d` não divida nem `*m` nem `*n`, incrementa-se `d` e repete-se o processo. O algoritmo termina quando `m` e `n` foram reduzidos a 1, momento em que todos os seus fatores primos (com multiplicidade) já foram extraídos e incorporados ao acumulador, que corresponde, portanto, ao MMC entre os valores originais.

A seguir, o Código 1 apresenta uma versão em linguagem C do algoritmo, servindo de ponte entre a descrição acima e a implementação em Assembly apresentada na Seção **??**.

```
1 int divide(int *m, int *n, int d) {
2     int dividiu = 0;
3
4     if (*m % d == 0) {
```

```
        *m /= d;
        dividiu = 1;
    }
    if (*n % d == 0) {
        *n /= d;
        dividiu = 1;
    }

    return dividiu;
}

int MMC(int m, int n) {
    int result = 1;
    int d = 2;

    while (m != 1 || n != 1) {
        if (divide(&m, &n, d)) {
            result *= d;
        } else {
            d++;
        }
    }

    return result;
}

int main() {
    int m, n;
    scanf("%i", &m);
    scanf("%i", &n);

    int mmc = MMC(m, n);

    printf("O MMC e: %i\n", mmc);
    return 0;
}
```

Listing 1: Pseudocódigo em C do algoritmo

Vale comentar que uma possível otimização para a estratégia adotada seria testar apenas divisores primos, em vez de verificar todos os inteiros sequencialmente. Isso porque não faz sentido, testar divisores compostos, como por exemplo 6, já que seus fatores primos (2 e 3) necessariamente teriam sido testados anteriormente. Dessa forma, poderia ser mantido um vetor contendo números primos, e a variável `d` passaria a assumir apenas esses valores, reduzindo a quantidade de testes desnecessários e tornando o processo de fatoração mais eficiente, principalmente para entradas de maior magnitude.

3 Código em Assembly desenvolvido

O programa foi implementado em MIPS Assembly e é organizado em três sessões principais: a rotina principal (`main`), responsável pela leitura dos dados e impressão do resultado; a sub-rotina `mmc`, que implementa o laço de cálculo do Mínimo Múltiplo Comum por divisões sucessivas; e a sub-rotina `divide`, chamada repetidamente por `mmc`, que tenta dividir `m` e/ou `n` por um divisor candidato `d`.

A Tabela 1 resume o papel de cada registrador utilizado, seguindo a convenção de uso da arquitetura MIPS (registradores `$t` como temporários não preservados entre chamadas, `$s` como registradores salvos/persistentes, `$a` para argumentos e `$v` para valores de retorno).

Registrador	Papel no programa
`$s0`	Acumulador do MMC (persiste durante todo o laço)
`$s1`	Divisor candidato `d` (persiste durante todo o laço)
`$a0, $a1`	Endereços (&m, &n) passados para `divide`
`$a2`	Valor do divisor `d` passado para `divide`
`$v0`	Código de *syscall* / valor de retorno de `divide` (0 ou 1)
`$t0, $t1, $t2`	Temporários intermediários (dividendo, resto, quociente)
`$ra`	Endereço de retorno da sub-rotina `divide`

Tabela 1: Registradores utilizados e seus papéis

3.1 Bloco principal (`main`)

O bloco `main` é responsável apenas pela inicialização da execução do programa. Ele realiza a leitura dos dois inteiros `m` e `n` por meio de chamadas de sistema, inicializa o acumulador do MMC em `$s0 = 1` e o divisor candidato inicial em `$s1 = 2`. Após essas inicializações, prepara os registradores necessários e transfere o controle para a sub-rotina `mmc`, que passa a executar o algoritmo principal.

```
main:
        # Imprime string
        li $v0, 4
        la $a0, msg_m
        syscall

        # Le inteiro m
        li $v0, 5
        syscall
        sw $v0, m
```

```
11
12          # Imprime string
13          li $v0, 4
14          la $a0, msg_n
15          syscall
16
17          # Ler inteiro n
18          li $v0, 5
19          syscall
20          sw $v0, n
21
22          li $s0, 1 # $s0 = mmc = 1
23          li $s1, 2 # $s1 = d = 2
24
25          # vai para mmc
```

Listing 2: Bloco `main`

3.2 Sub-rotina `mmc`

A sub-rotina `mmc` implementa o laço principal responsável pelo cálculo do MMC. Em cada iteração, verifica se os valores de `m` e `n` já foram reduzidos a `1`; caso isso ocorra, todos os fatores necessários já foram extraídos e o programa desvia para o rótulo `imprime`, onde o resultado armazenado em `$s0` é exibido. Caso contrário, a sub-rotina chama `divide` utilizando o divisor candidato atual. Se a divisão de pelo menos um dos dois valores ocorrer (`$v0` diferente de zero), o divisor é incorporado ao acumulador por meio de uma multiplicação e a próxima iteração reutiliza o mesmo valor de `d`. Se nenhuma divisão for realizada, o fluxo segue para o rótulo `nao_dividiu`, onde o divisor candidato é incrementado antes do início da próxima iteração.

```
1  mmc:
2          # carrega m e n
3          lw $t0, m # $t0 = m
4          lw $t1, n # $t1 = n
5
6          # $t2 = minimoMMC = 1
7          li $t2, 1
8
9          # se m!=1 continua tentando dividir
10         bne $t0, $t2, tentaDividir
11
12         # A essa ponto do codigo pelo menos m eh 1
13         # mas talvez n ainda pode ser divisivel
14
15         # se n!=1 continua tentando dividir
```

```
16          bne $t1, $t2, tentaDividir
17
18          j imprime
19
20  tentaDividir:
21          la $a0, m           # $a0 = &m
22          la $a1, n           # $a1 = &n
23          move $a2, $s1         # $a2 = d
24
25          jal divide
26
27          beqz $v0, nao_dividiu # se não dividiu ninguém, tenta
                outro d.
28
29          mul $s0, $s0, $s1 # caso contrário, pelo menos um deles
                foi dividido. Multiplicar o MMC pelo d atual.
30          j mmc
31
32  nao_dividiu:
33          addi $s1, $s1, 1 # d++
34          j mmc
35
36  imprime:
37          # imprime string de msg MMC
38          li $v0, 4
39          la $a0, msg_mmc
40          syscall
41
42          # imprime string do MMC
43          li $v0, 1
44          move $a0, $s0
45          syscall
46
47          # acaba o programa ;D
48          li $v0, 10
49          syscall
```

Listing 3: Sub-rotina `mmc`

3.3 Sub-rotina `divide`

A sub-rotina `divide` recebe os endereços das variáveis `m` e `n` nos registradores `$a0` e `$a1`, respectivamente, e o divisor candidato `d` em `$a2`. Inicialmente, verifica se `m` é divisível por `d`; quando o resto da divisão é zero, substitui o valor de `m` pelo quociente obtido e registra que uma divisão ocorreu por meio de `$v0 = 1`. Em seguida, o mesmo procedimento é realizado para `n`, de forma independente do resultado da primeira verificação. Ao término da execução,

o valor retornado em `$v0` indica se pelo menos uma das duas divisões foi efetuada, informação utilizada pela sub-rotina `mmc` para decidir se o divisor atual deve ser reutilizado ou incrementado.

```
divide:
        li $v0, 0 # retorno eh zero ate que se prove o contrario

        # testa *m
        lw $t0, 0($a0) # $t0 = *m
        div $t0, $a2
        mfhi $t1 # resto

        bnez $t1, testa_n # se resto != 0, não divide

        mflo $t2 # quociente
        sw $t2, 0($a0) # *m = *m / d
        li $v0, 1  # retorna 1 pois dividiu

testa_n:
        lw $t0, 0($a1) # $t0 = *n
        div $t0, $a2
        mfhi $t1 # resto

        bnez $t1, fim_divide # se resto != 0, termina

        mflo $t2 # quociente
        sw $t2, 0($a1) # *n = *n / d
        li $v0, 1 # dividiu

fim_divide:
        jr $ra
```

Listing 4: Sub-rotina `divide`

4 Explicação detalhada das instruções utilizadas no código feito pela equipe

Muito texto.